

# A BDI-LLM Hybrid Multi-Agent System for DeliverooJS

Davide Senette

email: [davide.senette@studenti.unitn.it](mailto:davide.senette@studenti.unitn.it), mat: 267070

Davide Sarti

email: [davide.sarti@studenti.unitn.it](mailto:davide.sarti@studenti.unitn.it), mat: 267894

Autonomous Software Agents Course

17 June 2026

## 1 System Overview

**Agent A — the LLM agent.** Runs an *embedded* autonomous BDI loop (`startAutonomousBDI`) identical in implementation to Agent B’s, and additionally listens on the chat channel. Each incoming chat message is interpreted by an LLM and executed through a tool-call loop (Section 6). The embedded BDI keeps harvesting parcels with zero model calls while the LLM merely reads or queries state; it is paused only when a tool is about to take exclusive control of the agent’s body (movement, delegated collection, rendezvous).

**Agent B — the BDI agent.** A pure reactive BDI agent with no LLM in its loop. It scores and executes intentions autonomously (Section 4) and can be *preempted* by directives sent over the wire by Agent A (Section 7).

Both agents instantiate `createBeliefState()` (Section 2) independently; the two belief states are not a shared memory region but are kept consistent by an explicit synchronization channel for persistent rules. The design separates four concerns that recur throughout the codebase:

- **Deliberation** — option generation and scoring (`bdi/options.js`, `bdi/bdiAgent.js`), independent of how an intention is eventually carried out.
- **Action execution** — the plan layer (`actions/actions.js`), which turns a predicate into pathfinding and game-socket calls, falling back to PDDL when needed.
- **Coordination** — a directive/status protocol (`bdi/coordination.js`, `llm/coordinator.js`) layered on top of, but decoupled from, the scoring loop.
- **Shared beliefs** — one schema (`beliefs/beliefState.js`) consumed identically by scoring, navigation, the LLM tools and the PDDL problem builder.

Figure 1 shows the resulting data and control flow.

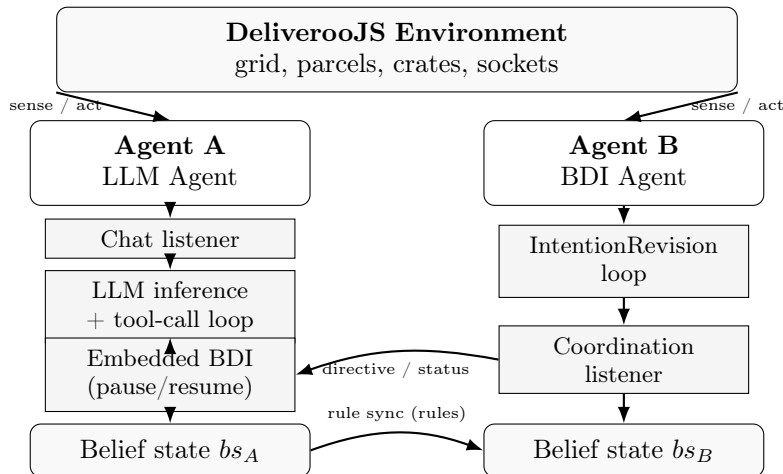


Figure 1: Overall system architecture: two agents with structurally identical belief states, connected by a rule-synchronization channel and a directive/status coordination channel. The LLM agent’s embedded BDI is the same code path as the standalone BDI agent’s loop.

## 2 Belief State

`beliefs/beliefState.js` defines `createBeliefState()`, the single source of truth read by option generation, scoring, navigation, the LLM tools and the PDDL problem builder. No module keeps a parallel copy of world state; everything is derived from this object on demand. Table 1 summarizes its top-level sections.

| Section                                                                             | Content                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>me</code>                                                                     | <code>{id, name, x, y, score}</code> — identity and current position.                                                                                                                                                                                                                                                                                                           |
| <code>parcels</code> , <code>crates</code> , <code>agents</code> , <code>map</code> | Maps of sensed world objects keyed by id, refreshed on every sensing event.<br><code>tiles</code> , <code>deliveryTiles</code> , <code>spawnTiles</code> , <code>pushableTiles</code> (type-5 / type-“5!” tiles a crate can be pushed onto), <code>deliveryDistanceMap</code> , <code>spawnDistanceMap</code> (precomputed BFS distance tables, Section 5), <code>grid</code> . |
| <code>config</code>                                                                 | <code>observationDistance</code> , <code>movementDuration</code> , <code>playerCapacity</code> , <code>parcelDecayingEvent</code> , <code>parcelGenerationEvent</code> , <code>maxParcels</code> — server-declared parameters.                                                                                                                                                  |
| <code>timing</code>                                                                 | <code>decayTicks</code> (global counter of observed decay events) and <code>decayPerStep</code> (EMA of ticks per completed move, Section 4).                                                                                                                                                                                                                                   |
| <code>rules</code>                                                                  | Persistent strategy rules (Table 2), set only by the LLM agent’s tools but mirrored onto the BDI agent’s copy.                                                                                                                                                                                                                                                                  |
| <code>coordination</code>                                                           | <code>active</code> , <code>queue</code> , <code>current</code> , <code>waiting</code> , <code>sendStatus</code> , <code>lastActivityMs</code> — directive-mode state (Section 7).                                                                                                                                                                                              |
| <code>carry</code>                                                                  | <code>{count, campSteps}</code> — cached carried-parcel count and tiles patrolled while carrying in the current camp episode.                                                                                                                                                                                                                                                   |
| <code>lastParcelHint</code>                                                         | <code>{x, y, ts}</code> of the last tile where a parcel was <i>actually picked up</i> (not merely sighted); anchors camping.                                                                                                                                                                                                                                                    |

Table 1: Top-level sections of the belief state.

| Field                                                             | Semantics                                                                                                                                                                                         |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>stackSize</code>                                            | Array of <code>{mode, count, met, unmet}</code> ; several compatible constraints (e.g. <code>at_least 2</code> and <code>at_most 5</code> ) coexist; conflicting ones are resolved at write time. |
| <code>parcelValueRules</code>                                     | Array of <code>{minReward, maxReward, mult, delta}</code> value-band remaps applied at delivery time.                                                                                             |
| <code>penaltyDeliveries</code> , <code>preferredDeliveries</code> | <code>"x,y" → {penalty} / {reward}</code> maps for specific delivery tiles.                                                                                                                       |
| <code>deliveryMultipliers</code>                                  | <code>"x,y" → multiplier</code> map for specific delivery tiles.                                                                                                                                  |
| <code>penaltyTiles</code>                                         | <code>"x,y" → {penalty}</code> map consumed by A* as a soft cost ( <i>not</i> a hard block).                                                                                                      |
| <code>rendered</code>                                             | Human/LLM-readable string derived from the structured fields, kept current by the rule tools.                                                                                                     |
| <code>onChange</code>                                             | Optional hook (wired by the LLM agent) fired after any rule mutation, used to push a full rule snapshot to the partner.                                                                           |

Table 2: Fields of `bs.rules`. All are soft preferences: they shift a score but, by construction (Section 4), can never strand the agent without a viable delivery or a passable route.

**Object permanence.** Sensing is local and intermittent: a parcel or agent that leaves observation range simply stops appearing in updates. Several belief-update paths therefore treat the *absence* of an expected entity as negative evidence rather than as silence, combined with a short time-to-live: an agent’s last known position (`AGENT_MEMORY_TTL_MS`, 3s) is retained briefly for the race-discount and soft-obstacle computations and then dropped, so stale sightings do not bias scoring or pathfinding indefinitely. The same TTL discipline applies to the avoided-tile memory used during navigation (Section 5): a tile that just refused a move is hard-avoided only for a few move-durations, after which it becomes eligible again while the blocking agent is still in sensing range.

## 3 Option Generation

`optionsGeneration(agent, bs)` (`bdi/options.js`) turns the current belief state into a set of candidate predicates and inserts them into the intention queue in one batched call. It is purely generative: it decides what is *reachable*, never what is *worth doing* — valuation is the exclusive responsibility of `intentionScore` (Section 4).

`agent.pushBatch(options)` filters out predicates already in the failed-intention cooldown pool or already queued, inserts the rest, and re-sorts the queue exactly *once* regardless of batch size — avoiding  $O(n)$  re-sorts for  $n$  newly generated options.

`optionsGeneration` returns immediately when `bs.coordination.active` is true: in directive mode

| Predicate                                      | Generated when                                                                           | Source                                                                                      |
|------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>go_pick_up x y</code><br><code>id</code> | a free parcel has a finite pickup-route distance, and carried count < effective capacity | one per visible free parcel                                                                 |
| <code>go_drop_off x y</code>                   | at least one parcel is carried                                                           | one per delivery tile reachable from the current cell                                       |
| <code>camp</code>                              | <code>AGENT_CONFIG.behavior.camp</code> is enabled                                       | single option, validity decided entirely in scoring                                         |
| <code>explore</code>                           | always, as the idle fallback pushed directly by the loop                                 | not generated here; pushed by <code>IntentionRevision.loop()</code> when the queue is empty |

Table 3: Predicates produced by `optionsGeneration` (`explore` is pushed separately by the idle branch of the revision loop, Section 4).

the BDI executes only what the LLM commands, so ordinary scored options must neither compete with nor preempt the active coordination intention (Section 7).

## 4 Scoring and Intention Revision

### 4.1 The IntentionRevision loop

`IntentionRevisionRevise` (`bdi/bdiAgent.js`) maintains an intention queue and a failed-intention cooldown pool (predicate  $\rightarrow$  {`addedAtMs`, `cooldownMs`}, flat `FAILED_INTENTION_RETRY_MS` = 1s, no exponential backoff: a failure is almost always transient congestion). Each pass of `loop()`:

1. Idles if `paused` (an embedded owner, e.g. the LLM mission handler, is holding the body) or runs one coordination step if `coordination.active`.
2. Re-queues failed intentions whose cooldown has elapsed.
3. Drops the head intention if it has become structurally invalid (parcel already carried/gone, or no longer carrying anything for a queued delivery).
4. Otherwise `achieve()`s the head intention, classifying the outcome as *completed*, *preempted*, *failed* or *stopped*, and reacts accordingly (Section 4.4).
5. Pushes `explore` when the queue is empty.

### 4.2 Score computation

`intentionScore(predicate)` is the single authority on a predicate’s worth; every number below is computed fresh from `bs` at scoring time. The quantity represents the potential optimistic score gain from an action.

**Decay factor.** Reward loss is priced through one quantity, the EMA-estimated reward lost per carried parcel per tile traveled:

$$\text{factor} = \text{decayPerStep} = \alpha \cdot \text{ticks}_t + (1 - \alpha) \cdot \text{decayPerStep}_{t-1}, \quad \alpha = 0.15 \quad (1)$$

seeded by the config-derived prior `movementDuration/decayInterval` and sampled purely from server events (decay ticks observed between two consecutive move acknowledgements), so it is immune to client load, network latency or server lag: a slower step necessarily contains proportionally more ticks.

**Drop-off score.** For `go_drop_off x y` with route distance  $d$  (read from `deliveryDistanceMap`) and carried parcels  $p \in P$ :

$$\text{projected}_p = \max(0, \text{reward}_p - d \cdot \text{factor}) \quad (2)$$

$$\text{banked}_p = \text{projected}_p \cdot \text{mult}_v + \text{delta}_v \quad (\text{value-band remap, Section 4.2.1}) \quad (3)$$

$$\text{sumBanked} = -\text{DROP\_DISINCENTIVE} + \sum_{p \in P} \text{banked}_p \quad (4)$$

$$\text{scoreAfterStack} = \text{sumBanked} \cdot \text{mult}_s + \text{delta}_s \quad (\text{stack delivery modifier at } |P|) \quad (5)$$

$$S_{\text{drop}}(x, y) = \max(0, \text{scoreAfterStack} \cdot m(x, y) + r(x, y) - \pi(x, y)) \quad (6)$$

where  $m, r, \pi$  are the tile’s delivery multiplier, preferred reward and penalty (`adjustDeliveryScore`). `DROP_DISINCENTIVE` is currently 0.

**Pickup score.** For `go_pick_up` of a new parcel  $n$  with full route distance  $d$  (pickup leg + nearest-delivery leg) and carried parcels  $P$ :

$$\text{raceProb}(n) = \frac{1}{1 + \exp((d_{\text{me} \rightarrow n} - d_{\text{opp} \rightarrow n})/\tau)}, \quad \tau = \text{RACE\_DISCOUNT\_TAU} = 2 \quad (7)$$

$$\text{newDelivered} = \left( \max(0, \text{reward}_n - d \cdot \text{factor}) \cdot \text{mult}_v + \text{delta}_v \right) \cdot \text{raceProb}(n) \quad (8)$$

$$\text{carriedDelivered} = \sum_{p \in P} \left( \max(0, \text{reward}_p - d \cdot \text{factor}) \cdot \text{mult}_{v,p} + \text{delta}_{v,p} \right) \quad (9)$$

$$S_{\text{pick}}(n) = (\text{carriedDelivered} + \text{newDelivered}) \cdot \text{mult}_s(|P|+1) + \text{delta}_s(|P|+1) \quad (10)$$

The stack modifier in Eq. 10 is evaluated *optimistically* at the resulting count  $|P|+1$ : gathering toward a target is priced with the rule’s `met` modifier, and only a pickup that *overshoots* an `exactly/at_most` cap takes the `unmet` modifier (`stackPickupModifier`), so a sufficiently rich parcel can still justify breaking the target stack. The race discount is applied *after* the value-band remap, never before: discounting first could push a high-value parcel into a “worth 0” band the moment any agent contests it.

**Camp score.** `camp` is valid only while carrying ( $|P| > 0$ ), the harvest hint is “hot”, capacity remains, and no delivery tile is within `CAMP_NEAR_DELIVERY_TILES` = 8 (loitering near a delivery is never worth it: quick cycles win). Patience is primarily spatial, then scaled by the server’s declared spawn rate:

$$\text{patienceMs} = \text{clamp}(\mu \cdot 1000 \cdot (1.37^{n_{\text{adj}}} - 1), 0, \mu \cdot 8000), \quad n = \min(n_{\text{adj}}, 10) \quad (11)$$

forced to 0 when  $n_{\text{adj}} < 4$  or the server declares infinite parcel generation, with  $n_{\text{adj}}$  the number of spawn tiles within Manhattan radius 3 of the harvest anchor (the constants are `CAMP_PATIENCE_MIN_TILES`=4 and `CAMP_PATIENCE_SATURATION_TILES`=10) and  $\mu = \min(\text{CAMP\_SPAWN\_RATE\_MAX\_MULT}, \max(1, \text{CAMP\_SPAWN\_RATE\_NEUTRAL\_MS}/\text{parcelGenerationEvent}))$ , `CAMP\_SPAWN\_RATE\_MAX\_MULT`=1.3 and `CAMP\_SPAWN\_RATE\_NEUTRAL\_MS`=2000ms, so a map that spawns faster than the 2s neutral rate stretches both the growth curve and its ceiling, making a hot pocket worth camping longer. While hot, the camp loss budget bounds how long camping is worth it relative to delivering now:

$$\text{horizon} = \frac{\max(\text{CAMP\_LOSS\_BUDGET\_FRACTION} \cdot \text{totalReward}, \text{stackCampGain})}{\text{factor} \cdot |P|} \quad (12)$$

where `stackCampGain` is the extra delivery value reachable by gathering more parcels up to an active stack-rule target (0 when no rule, or when the cap is already exceeded and cannot be undone by waiting). `Camp` is invalid once `campSteps`  $\geq$  horizon. When valid:

$$S_{\text{camp}} = \text{totalReward} + \text{CAMP\_INCENTIVE}, \quad \text{CAMP\_INCENTIVE} = 0.02 \quad (13)$$

which deliberately outranks a plain delivery of the same load (so the agent gathers a fuller load first) but stays below any real pickup ( $\text{totalReward} + \text{reward}_n$ ), so a visible parcel is still grabbed before camping is reconsidered.

**Exploration score.** A flat floor,  $S_{\text{explore}} = \text{EXPLORATION\_INCENTIVE} = 0.01$ , strictly below camp so an idle agent prefers to camp a hot pocket over wandering, but high enough to keep the agent moving when nothing else scores positively.

#### 4.2.1 Persistent-rule modifiers

`bdi/ruleScoring.js` implements the rule effects referenced above as pure functions of `bs.rules`, kept independent of the scoring loop so they cannot silently diverge from it:

- **Parcel value bands.** Each rule `{minReward,maxReward,mult,delta}` contributes its modifier when a value  $v$  lies in  $[\text{minReward}, \text{maxReward}]$  (open bounds), combined multiplicatively/additively across overlapping rules:  $v' = v \cdot \prod \text{mult}_i + \sum \text{delta}_i$ .
- **Stack-size combination.** `combineStack` multiplies every active rule’s chosen modifier (`met` or `unmet`, selected per-context) and sums the deltas; two rules *conflict* (and the older is dropped on write) exactly when their satisfied-count ranges —  $[N, \infty)$  for `at_least`,  $[1, N]$  for `at_most`,  $[N, N]$  for `exactly` — do not overlap.
- **Delivery-tile preferences.**  $\text{adjusted} = \max(0, \text{base} \cdot m + r - \pi)$  (Eq. 6); the floor at 0 is what keeps a penalty a pure *reordering* device rather than a way to forbid the agent’s only path to banking reward.

Table 4 collects the numeric constants governing scoring and preemption.

| Constant                  | Value   | Role                                                                    |
|---------------------------|---------|-------------------------------------------------------------------------|
| HOARD_CAP                 | 10      | Hard ceiling on carried parcels regardless of server-declared capacity. |
| RACE_DISCOUNT_TAU         | 2 tiles | Sigmoid temperature in Eq. 7.                                           |
| PREEMPTION_HYSTERESIS     | 0.25    | Relative margin a challenger must clear (Section 4.3).                  |
| PREEMPTION_EPSILON        | 0.001   | Additive margin guarding against float-equality ties.                   |
| CAMP_NEAR_DELIVERY_TILES  | 8       | Distance below which camping is disabled in favor of delivering.        |
| CAMP_LOSS_BUDGET_FRACTION | 0.05    | Fraction of carried reward camping may bleed to decay.                  |
| EXPLORATION_INCENTIVE     | 0.01    | Flat exploration floor.                                                 |
| CAMP_INCENTIVE            | 0.02    | Flat margin by which camp outranks an equal-value delivery.             |
| FAILED_INTENTION_RETRY_MS | 1000 ms | Flat cooldown before a failed predicate is re-probed.                   |

Table 4: Scoring and preemption constants (`utils/constants.js`, `bdi/options.js`).

### 4.3 Hysteresis preemption

`sortQueueByScore()` re-scores every queued intention, drops any with score  $\leq 0$  (so a hard-forbidden delivery, floored to 0 by Eq. 6, naturally leaves the queue), and re-sorts by descending score. Switching the *running* intention is not free — a half-walked route has a real sunk cost — so a challenger must clear a hysteresis band:

$$S_{\text{challenger}} > \begin{cases} S_{\text{running}} (1 + h) + \varepsilon & \text{if } S_{\text{running}} > 0 \\ 0 & \text{if } S_{\text{running}} \leq 0 \end{cases} \quad (14)$$

with  $h = 0.25$  and  $\varepsilon = 0.001$  (Table 4); an already-invalid running intention ( $S_{\text{running}} \leq 0$ ) is abandoned immediately. This protects deliveries in final approach naturally: the drop-off score in Eq. 6 *peaks* right before the tile (route distance  $\rightarrow 0$ ), so the bar for interruption is highest exactly where switching would waste the most progress. A `committedManeuver` flag (set while a PDDL crate-push is in flight, Section 8) unconditionally disables preemption: restarting a partial push from scratch would waste the detour already taken to get behind the crate.

When preemption fires, the running intention is `stop(STOP_REASON_PREEMPTION)`’d; its `achieve()` promise rejects with a tagged *preempted* error, and the loop **requeues** a fresh `Intention` for the same predicate (not failed, not cooled down) so it competes again on the next sort.

### 4.4 Failure handling

| Outcome          | Handling                                                                                                                                                                                                                                                                                                                       |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>completed</i> | Intention removed from the queue; no special action.                                                                                                                                                                                                                                                                           |
| <i>preempted</i> | Re-queued as a fresh intention (Section 4.3); never enters the cooldown pool.                                                                                                                                                                                                                                                  |
| <i>failed</i>    | Added to the failed-intention pool for <code>FAILED_INTENTION_RETRY_MS</code> ; if the failed predicate was <code>go_drop_off</code> , <code>#pushNextNearestDelivery</code> immediately queues the next-nearest <i>other</i> delivery tile so the agent is never left with zero delivery options while parcels keep decaying. |
| <i>stopped</i>   | A clean, non-preemption stop (e.g. a manual pause or a propagated <code>["stopped"]</code> ); removed without cooldown or replacement.                                                                                                                                                                                         |

Table 5: Intention outcomes and their effect on the queue.

Figure 2 summarizes the full cycle.

## 5 Plans and Navigation

`executePredicate(actions/actions.js)` is the one entry point translating a predicate into game-socket calls; it is shared verbatim by the autonomous loop, the coordination layer and the LLM’s delegated tools.

**Navigation principles.**

- **A\* with mixed hard/soft obstacles.** Walls and crates are hard; other agents are hard only within `SOFT_OBSTACLE_HARD_RADIUS=1` tiles of the start (they are genuinely in the way *now*), and merely add `AGENT_SOFT_PENALTY=7.5` to the edge cost otherwise (they will likely have moved by the time the agent arrives).
- **Soft penalties from rules.** Entering a tile in `bs.rules.penaltyTiles` costs extra (added to `BASE_STEP_COST=1`) but is never refused, so an LLM navigation rule can steer the route without ever stranding the agent.

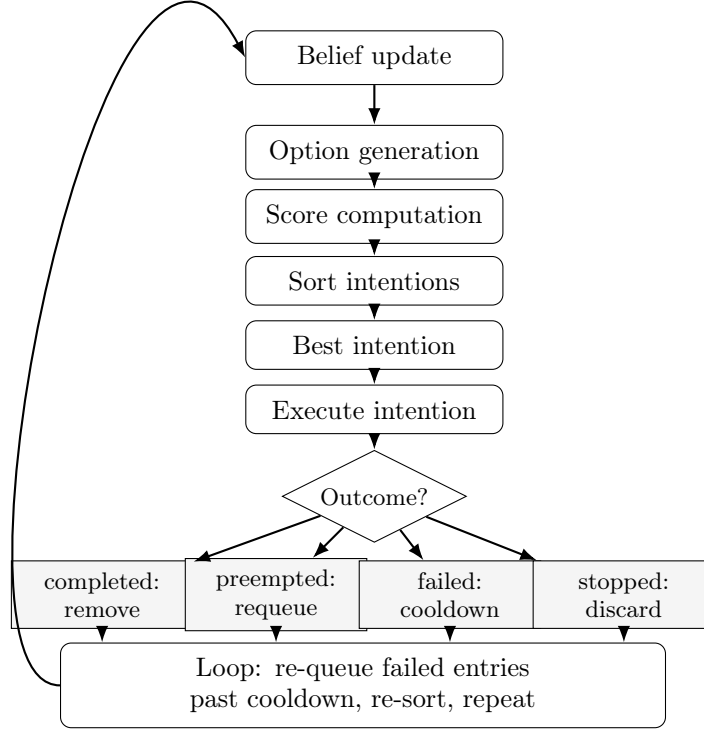


Figure 2: Intention revision cycle. The sort step (Eq. 14) is where hysteresis preemption is decided; the four outcomes feed back into the same loop via the failed-intention pool and the delivery fallback of Section 4.4.

| Predicate                          | Behavior                                                                                                                                                                                           |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>go_to x y</code>             | A*-routes to $(x, y)$ , replanning on a blocked step (up to <code>MAX_REPLANS=3</code> ) and falling back to PDDL when a crate is the sole blocker.                                                |
| <code>go_pick_up x y id</code>     | <code>go_to</code> then <code>pickup()</code> ; records <code>lastParcelHint</code> on a successful grab.                                                                                          |
| <code>go_drop_off x y</code>       | <code>go_to</code> then <code>putdown()</code> ; increments <code>bs.metrics.deliveredParcels</code> only when the tile is a genuine delivery tile (a handoff drop on a plain tile banks nothing). |
| <code>explore</code>               | <code>go_to</code> on the least-recently-visited reachable spawn tile ( <code>findCellsToExplore</code> , distance/staleness weighted).                                                            |
| <code>camp</code>                  | Patrols the perimeter of the spawn cluster around the harvest anchor (Section 4), charging <code>carry.campSteps</code> .                                                                          |
| <code>go_near x y maxDist</code>   | <code>go_to</code> on the closest reachable tile inside the Manhattan diamond of radius <code>maxDist</code> around $(x, y)$ ; used by rendezvous.                                                 |
| <code>wait signal timeoutMs</code> | Blocks until <code>bs.coordination.waiting</code> is cleared by an incoming <code>signal</code> message, or until an optional timeout; default is unbounded.                                       |

Table 6: Plan predicates dispatched by `executePredicate`.

- **Adaptive timeout.** `timeout = max(GO_TO_TIMEOUT_MS, |path| · movementDurationMs · GO_TO_TIMEOUT_SAFETY_FACTOR)`, with `GO_TO_TIMEOUT_SAFETY_FACTOR=3`, so short legs fail fast and long legs are not spuriously timed out.
- **avoidTiles.** A tile that just physically refused a move is hard-blocked for `AVOID_TENURE_MOVE_FACTOR=3` move-durations, forcing the replanner onto a genuinely different route instead of greedily re-entering a proven choke; the memory expires quickly so a cleared corridor is retried while still in sensing range.
- **Opportunistic pickup and delivery.** `opportunisticActions()` runs after every step: it grabs a free, rule-eligible parcel standing on the just-entered tile (zero detour) and auto-banks carried parcels on a delivery tile, unless a stack rule is still unmet or the tile is penalised — suspended entirely while `coordination.active`.
- **BFS distance maps.** `deliveryDistanceMap` and `spawnDistanceMap` are precomputed once per tile (multi-source shortest-path tables) so route-distance lookups used throughout scoring (Eqs. 6–10) are  $O(1)$  instead of re-running BFS/A\* on every score evaluation.
- **Crate detection.** `crateBlocksRoute()` plans twice — once respecting crates, once with `ignoreCrates:true` — and concludes a crate is the blocker only when the first fails and the second succeeds, avoiding false positives from an agent merely orbiting a temporarily occupied choke.
- **PDDL invocation.** On a confirmed crate block, `pddlReach` sets `bs.committedManeuver=true`, solves and executes a push plan (Section 8), and clears the flag in a `finally`, regardless of outcome.

## 6 The LLM Layer

### 6.1 Architecture

`startLLMAgent(socket, bs, llmState, actions)` wires three independent pieces onto one agent: an embedded autonomous BDI (`startAutonomousBDI`, identical code path to Agent B), a `createCoordinator` for talking to the partner, and a chat listener. Incoming chat messages are pushed through `createMissionQueue`, which serializes mission handling so two rapid messages never race over the same pause/resume state or the same parcels.

Pausing is *lazy*: the embedded BDI keeps harvesting while the LLM reasons about a message, and is paused only the first time a body-controlling tool (Table 7) is about to execute, via `pauseBdiForMission()` (idempotent, guarded by `pausedForMission`). A pure rule/query mission therefore never interrupts on-going play. The `finally` block unconditionally calls `bdi.resume()` at mission end, *unless* the agent was deliberately parked on an external signal (`partnerParkedOn` set, Section 7), in which case the next mission turn must relay that signal before resuming.

| Pauses the embedded BDI               |                               | Does not pause it                 |                             |
|---------------------------------------|-------------------------------|-----------------------------------|-----------------------------|
| <code>collect_and_deliver,</code>     | <code>go_to,</code>           | <code>observe_environment,</code> | <code>calculate,</code>     |
| <code>go_pick_up,</code>              | <code>go_drop_off,</code>     | every rule/navigation             | store tool,                 |
| <code>rendezvous_with_partner,</code> |                               | <code>direct_partner,</code>      | <code>signal_partner</code> |
| <code>handoff_to_partner,</code>      | <code>wait_for_partner</code> |                                   |                             |

Table 7: BDI\_PAUSING\_TOOLS: only tools that take exclusive control of this agent’s own body (or must hold it still for a teammate) pause the embedded loop.

### 6.2 Prompt engineering

The system prompt (`SYSTEM_EXECUTOR_PROMPT`) is organized as a strict decision procedure, not free-form instructions:

**Role / output contract.** Exactly one tool call per step; the loop continues until `final_reply`.

**Reward phrasing.** Any mission tying points to an outcome (“+N pts if X”) is reframed as motivation, never as a passive fact: it resolves either to a durable RULE or to an active TASK, never to a no-op.

**Rejection whitelist.** A *closed* list of four grounds — unresolved coordinate placeholder, explicit negative immediate reward, violation of an active persistent rule, or genuinely garbled text. Everything else must be classified and acted on; the prompt explicitly forbids inventing further rejection reasons (vagueness, missing specifics, future tense).

**Anti-hallucination.** The model may only use parcels/tiles/rewards present in the supplied state or a fresh `observe_environment` call — never invented.

**Mission classification.** Every mission is first sorted into RULE, TASK or QUERY; the prompt calls this “the most common mistake” and forces the decision before any tool call.

**Compound missions.** A boolean `more` flag on terminal tools lets “do X AND do Y” missions chain multiple terminal calls without ending after the first clause.

**Team coordination rules.** Eight numbered rules disambiguating solo vs. team missions, when to use `rendezvous_with_partner` versus manual `direct_partner` + `wait_for_partner`, and the asymmetric handoff protocol (Section 7).

**Few-shot examples.** `EXECUTOR_EXAMPLES` enumerates ~20 worked mission→tool-sequence pairs covering rejections, factual queries, durable rules (including compound and replacement cases), harvesting tasks, rendezvous and handoffs — each annotated with the reasoning that produced the sequence.

### 6.3 Tool catalog

| Category         | Tools                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Self-action      | <code>move_to</code> , <code>pick_up_parcel</code> , <code>deliver_carried_parcel</code> ,<br><code>collect_and_deliver</code> (delegates to the embedded BDI,<br>Section 6.4)                                                                                                                                                                                                                                                                                                       |
| Query            | <code>observe_environment</code> , <code>calculate</code> , <code>resolve_delivery_tile</code>                                                                                                                                                                                                                                                                                                                                                                                       |
| Coordination     | <code>rendezvous_with_partner</code> , <code>handoff_to_partner</code> ,<br><code>direct_partner</code> , <code>signal_partner</code> , <code>wait_for_partner</code>                                                                                                                                                                                                                                                                                                                |
| Persistent rules | <code>set_stack_size_rule</code> , <code>remove_stack_size_rule</code> ,<br><code>set_parcel_value_rule</code> , <code>remove_parcel_value_rule</code> ,<br><code>forbid_delivery_tile</code> , <code>prefer_delivery_tile</code> ,<br><code>set_delivery_tile_multiplier</code> , <code>remove_delivery_tile_rule</code> ,<br><code>clear_durable_rules</code> , <code>block_navigation_tile</code> ,<br><code>unblock_navigation_tile</code> , <code>clear_navigation_rules</code> |

Table 8: Tool catalog exposed to the executor LLM (`SYSTEM_EXECUTOR_TOOLS`).

### 6.4 Execution loop

For each chat message, a fresh `[system, user]` message pair is built — the system prompt plus a user prompt embedding the raw mission text, the rendered rule set, and a validator snapshot of the current game state. The loop (capped at `MAX_ITERATIONS=100`) repeats:

1. Call the model with `temperature=0` and the full tool list; strip the control-flow fields `thought` and `more` before forwarding parameters to the tool implementation.
2. `mapExecutorAction` translates the executor’s tool name/shape into the internal action used by `executeTool` (e.g. `pick_up_parcel` → `go_pick_up`).
3. `validateActionAgainstPersistentRules` checks the action against active rules *before* execution; a violation is fed back as a tool observation so the model can pick a different step, without ever running the disallowed action.
4. If the action is in `BDI_PAUSING_TOOLS`, pause the embedded BDI (once).
5. Execute the tool, append its observation as a `tool` message.
6. **Terminal tools** (every persistent-rule/navigation store, plus `collect_and_deliver` via `toolResult.terminal`) end the mission immediately on success, using the tool’s own confirmation text as the chat reply — saving the extra round-trip a separate `final_reply` would cost — *unless* the tool errored or `more:true` was set for a remaining compound clause.
7. `final_reply` always ends the mission with an explicit message.

Concretely, for “collect three parcels and deliver them”: the chat message enters the mission queue, the LLM emits a single `collect_and_deliver(parcel=3)` tool call, the embedded BDI is unpaused and watches `deliveredParcels` climb with zero further model calls, and once the target is reached (or the time budget elapses) the tool returns and the BDI is re-paused, ending the mission in one model round-trip.

## 7 Multi-Agent Coordination

Coordination is a thin asynchronous protocol over the SDK’s `say` channel (`utils/coordProtocol.js`), layered on top of, but not replacing, each agent’s ordinary intention queue.

**Coordinator side** (`llm/coordinator.js`). `pendingWaiters` (`cid` → `resolver`) and `bufferedStatuses` (`cid` → `status`) together make `directPartner`/`waitForPartner` order-independent: a status that arrives before anyone waits for it is buffered and consumed by the next matching `waitForPartner`. `syncRules` fires automatically from `bs.rules.onChange`, so the BDI-only partner’s copy of `bs.rules` never drifts from the LLM’s.

**BDI side** (`bdi/coordination.js`). `setupBdiCoordination` attaches a message listener that pushes di-

| Message   | Direction | Payload / effect                                                                                                                         |
|-----------|-----------|------------------------------------------------------------------------------------------------------------------------------------------|
| directive | LLM→BDI   | {cid, command, args}; queued and, on arrival, preempts the BDI's running intention via <code>preemptForCoordination()</code> .           |
| status    | BDI→LLM   | {cid, ok, detail}; resolves the matching <code>wait_for_partner</code> , or is buffered if none is waiting yet.                          |
| signal    | LLM→BDI   | {signal}; out-of-band release of <code>bs.coordination.waiting</code> .                                                                  |
| rules     | LLM→BDI   | Full serialized rule snapshot (idempotent; a missed update self-heals on the next change), applied via <code>applyRulesSnapshot</code> . |

Table 9: Coordination protocol messages (COORD\_PROTO="coord").

rectives into `bs.coordination.queue`, flips `active`, and calls `agent.preemptForCoordination()`. `#runCoordination()` then drains the queue one directive at a time: `coordToPredicate` maps the wire command onto the same predicates Table 6 describes so directive execution reuses the ordinary plan layer verbatim. `vacateAfterDrop()` steps the agent one tile off its cell after a `putdown` so the partner can physically occupy the just-vacated tile to collect the handoff parcel. A `resume` directive clears `coordination.active`, returning control to the scored loop; an idle TTL (`COORD_RESUME_IDLE_TTL_MS=15s`) auto-resumes defensively if the LLM partner disappears mid-directive.

**Handoff protocol.** A cross-delivery handoff is driven end-to-end by a single tool, `handoff_to_partner`, rather than by the model manually sequencing `direct_partner/wait_for_partner` calls. The tool: (1) repeatedly issues `direct_partner(pickup)` with no coordinates so the BDI self-selects via `#selectBestPickup()` (reusing `intentionScore`, so it never diverges from normal play), stacking up to an optional target count or a safety cap (`HANDOFF_MAX_PICKUPS`); (2) sends `direct_partner(go_near)` to bring the loaded BDI adjacent to the LLM agent before dropping, so the hand-off lands within trivial reach; (3) issues `direct_partner(putdown)` at the BDI's own (now-adjacent) cell and waits for `status(ok)`, sent only after `vacateAfterDrop()` has stepped the BDI off the tile; (4) walks the LLM agent onto the vacated tile and retries the pickup a few times. (5) By default delivers the collected load to the nearest delivery tile and reports the parcels actually banked.

**Rendezvous / barrier synchronization.** `rendezvous_with_partner` sends `go_near` to the partner, moves the LLM agent itself in parallel, then waits for both arrivals before releasing the partner with `resume` — a single atomic barrier that avoids the race window a manual `direct/move/wait` sequence would leave open.

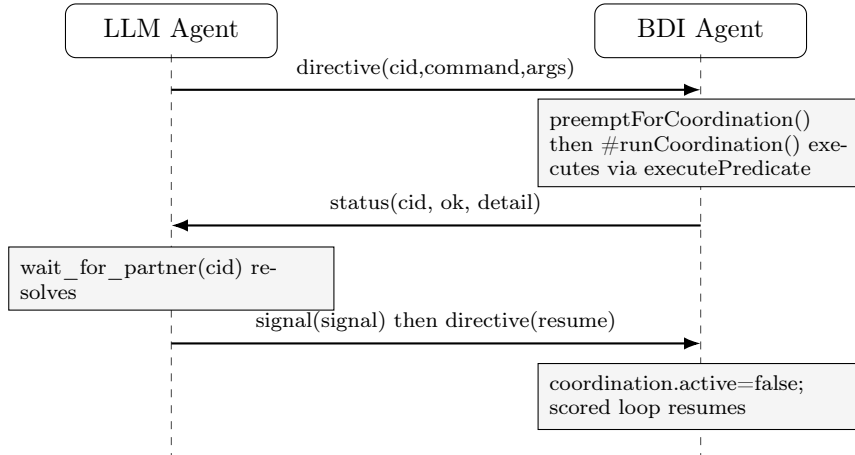


Figure 3: Multi-agent coordination protocol. Cross arrows are wire messages; boxed notes are local actions taken on receipt. The same exchange, with `pickup/putdown` commands, implements the asymmetric parcel handoff described above.

## 8 PDDL

When A\* reports that a route is blocked and `crateBlocksRoute()` confirms a crate is the sole obstacle, the agent falls back to a STRIPS planner (`pddl/domain.pddl`) capable of pushing crates aside — a maneuver A\* cannot express.

**Actions:** four `move-*` actions, one per direction (precondition: the adjacent tile in that direction is free), and four matching `push-*` actions, which move the agent onto the crate's tile while sliding the crate one step further in the same direction onto a `pushable` tile that is `free`. At execution time both collapse to the same primitive: the server has no separate push command, so moving *into* a crate that has a clear

| Predicate                                                              | Meaning                                                                     |
|------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| (tile ?t)                                                              | ?t is a walkable cell (walls, type 0, are excluded entirely).               |
| (free ?t)                                                              | No agent and no crate on ?t (parcels never block).                          |
| (delivery ?t),<br>(pushable ?t)                                        | Delivery zone / type-5 crate-receiving tile.                                |
| (up/down/left/right<br>?from ?to)                                      | Directional adjacency, emitted only between two existing tiles.             |
| (at-agent ?t)                                                          | Current agent position.                                                     |
| (parcel ?p),<br>(at-parcel ?p<br>?t), (carrying ?p),<br>(delivered ?p) | Parcel type guard, floor position, carried state, terminal delivered state. |
| (crate ?c), (at-crate<br>?c ?t)                                        | Crate type guard and position.                                              |

Table 10: Domain predicates (pddl/domain.pddl).

type-5 tile behind it pushes it automatically (planExecutor.js’s ACTION\_MAP maps every push-\* action to actions.move(direction)).

Listing 1: Push action (excerpt, pddl/domain.pddl): the agent and the crate must be co-linear in the push direction, and the destination must be both free and pushable.

```

1 (:action push-right
2   :parameters (?c ?agentPos ?cratePos ?destPos)
3   :precondition (and (crate ?c) (at-agent ?agentPos)
4     (at-crate ?c ?cratePos) (right ?agentPos ?cratePos)
5     (right ?cratePos ?destPos)
6     (free ?destPos) (pushable ?destPos))
7   :effect (and (at-agent ?cratePos)
8     (not (at-agent ?agentPos)) (free ?agentPos)
9     (at-crate ?c ?destPos) (not (at-crate ?c ?cratePos))
10    (not (free ?destPos)))
11 )

```

**Problem generation.** buildProblem(bs, goal) (pddl/problemBuilder.js) caches the static part of the map (tiles, adjacency, delivery/pushable sets) keyed on the bs.map.tiles array reference, rebuilding only when the server sends a new map; the dynamic part — occupancy, agent position, parcel and crate locations — is always recomputed from bs fresh. Tiles are named t\_<x>\_<y>; parcel and crate game ids (free-form strings) are hashed to a stable [a-z0-9] suffix with a djb2 variant (sanitize), since PDDL object names cannot contain arbitrary characters and the mapping never needs to be inverted — only uniqueness matters.

| Goal type                      | Generated conjunct(s)                                                                                                     |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| reach_tile {x,y}               | (at-agent t_x_y)                                                                                                          |
| deliver_parcel<br>{parcelId}   | (delivered p_<hash>)                                                                                                      |
| deliver_parcels<br>{parcelIds} | conjunction of (delivered ...) per id                                                                                     |
| free_tile {x,y}                | (free t_x_y) — used to ask “can a crate be moved off this tile”, the goal driving the crate-recovery maneuver of Figure 4 |

Table 11: Goal descriptors accepted by buildProblem.

**Planner invocation and fallback.** plan(bs, goal) races onlineSolver(domain, problem) against a SOLVER\_TIMEOUT\_MS=5s deadline; both an empty result and a timeout/solver error normalize to null, so the caller treats “no plan” and “solver unreachable” identically and falls back to ordinary A\*. While a plan is executing, bs.committedManeuver=true disables hysteresis preemption (Eq. 14): a crate push deliberately walks *away* from the target first (to get behind the crate), which would otherwise look like a worse intention and invite the revision loop to abandon it mid-push.

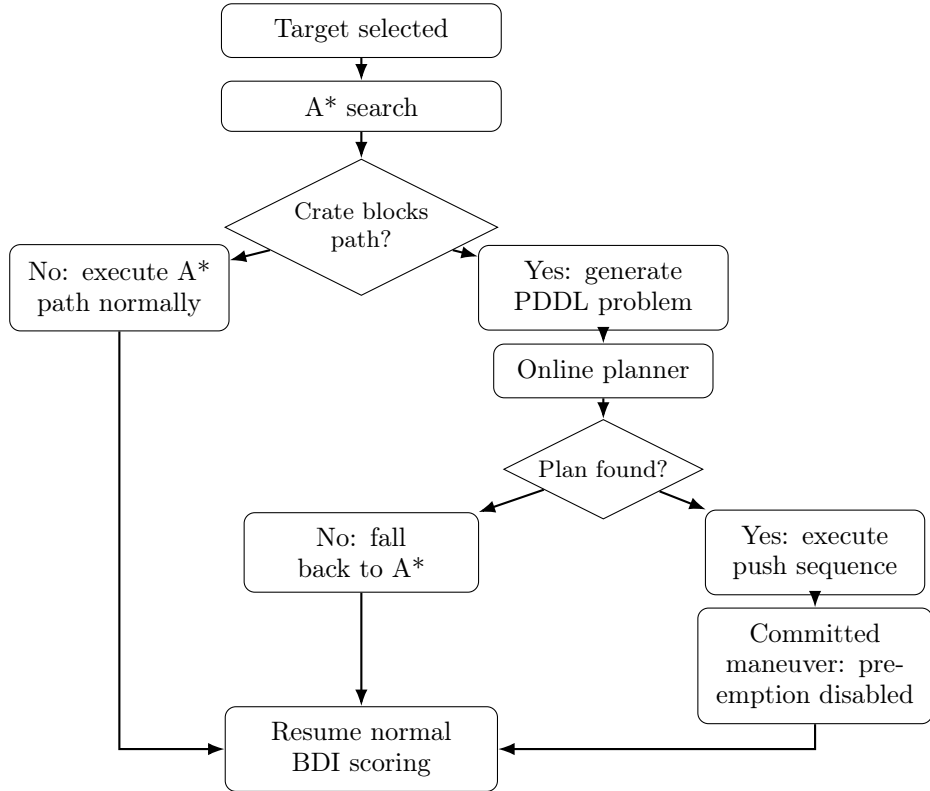


Figure 4: Crate blocking and PDDL recovery. `crateBlocksRoute()` double-checks the block (crate-respecting plan fails, crate-ignoring plan succeeds) before paying the cost of a solver call; a found plan runs as a committed maneuver immune to preemption.